

Lab Task 6: N-Grams Smoothing

Aim:

To implement N-Gram language modeling with smoothing techniques using Python, in order to handle zero probability issues in unseen word sequences.

Code:

```
import nltk

from nltk.util import ngrams

from collections import Counter

# Sample text
text = "natural language processing is fun and language processing is powerful"

# ♦ Tokenization
tokens = text.lower().split()

# ♦ Generate Bigrams
n = 2

ngrams_list = list(ngrams(tokens, n))

# Count frequencies
ngram_counts = Counter(ngrams_list)
unigram_counts = Counter(tokens)

# Vocabulary size
V = len(set(tokens))

# ♦ Laplace Smoothing Function
def laplace_prob(w1, w2):
    return (ngram_counts[(w1, w2)] + 1) / (unigram_counts[w1] + V)

print("Bigram Probabilities with Laplace Smoothing:\n")

# ♦ Print probabilities
for (w1, w2) in ngram_counts:
    prob = laplace_prob(w1, w2)
    print(f"P({w2} | {w1}) = {prob:.4f}")
```

Output:

Bigram Probabilities with Laplace Smoothing:

$$P(\text{language} \mid \text{natural}) = 0.2500$$

$$P(\text{processing} \mid \text{language}) = 0.3333$$

$$P(\text{is} \mid \text{processing}) = 0.3333$$

$$P(\text{fun} \mid \text{is}) = 0.2222$$

$$P(\text{and} \mid \text{fun}) = 0.2500$$

$$P(\text{language} \mid \text{and}) = 0.2500$$

$$P(\text{powerful} \mid \text{is}) = 0.2222$$